Universitatea Politehnica Timișoara
Facultatea de Automatică și Calculatoare

ITSC
16-bit General-Purpose Processor

Autor: Vasiliu Alexandru

Profesor coordonator: Bozdog Alexandru

Anul 3 Semestrul I

Subgrupa 3.2C

1. System Overview

The processor is a 16-bit Von Neumann style architecture with 2KB of memory. All ALU operations utilize an immediate 9-bit operand (extracted from the instruction and sign-extended to 16 bits)

Key Architecture Components

Word Size: 16-bit.

Memory: 1024 x 16-bit words (2KB total), addressed via a 10-bit address bus.

Control Unit: Microprogrammed, using a 7-bit Micro-Program Counter (uPC) and a 128-line Microcode ROM.

Registers: PC: Program Counter (16-bit)

IR: Instruction Register.

AC: Accumulator Register.

X & Y: Dual General-Purpose Registers selected by a reg_sel flag.

SP: Stack Pointer (16-bit, supports increment/decrement).

AR & DR: Address and Data Registers for memory interfacing.

Control Unit

The CPU control unit is microprogrammed; to help convert human readable microcode into binary a micro assembler was created in C. For

example, here is a portion of the microcode which implements the multiplication algorithm

```
"BOOTH_RADIX2"
    activate c15 ; A:=0, COUNTER:=0, Q[-1] := 0, M := X/Y, Q := sign_extend_unit_out[15:0]

"BR2_TEST"
    goto "BR2_ADD" if BOOTH01
    goto "BR2_SUB" if BOOTH10
"BR2_SHIFT"
    goto "TEST_COUNTER" activate c18

"BR2_ADD"
    goto "BR2_SHIFT" activate c16
"BR2_SUB"
    goto "BR2_SHIFT" activate c17

"TEST_COUNTER"
    goto "BR2_END" if count15
    goto "BR2_TEST" activate c19; counter increment
"BR2_END"
    activate c20; X/Y <- Q[15:0]
    goto "FETCH"
```

```
// --- BOOTH_RADIX2 (Address 0x31) ---
0x31 | 0010 | 0000 0000000 10000 | activate c15 ; A:=0, COUNTER:=0, Q[-1] := 0, M := X/Y, Q := sign_extend_unit_out[15:0]

// --- BR2_TEST (Address 0x32) ---
0x32 | 56A0 | 0101 0110101 00000 | goto "BR2_ADD" if BOOTH01
0x33 | 66C0 | 0110 0110110 00000 | goto "BR2_SUB" if BOOTH10

// --- BR2_SHIFT (Address 0x34) ---
0x34 | F6F3 | 1111 0110111 10011 | goto "TEST_COUNTER" activate c18

// --- BR2_ADD (Address 0x35) ---
0x35 | F691 | 1111 0110100 10001 | goto "BR2_SHIFT" activate c16

// --- BR2_SUB (Address 0x36) ---
0x36 | F692 | 1111 0110100 10010 | goto "BR2_SHIFT" activate c17

// --- TEST_COUNTER (Address 0x37) ---
0x37 | 7720 | 0111 0111001 00000 | goto "BR2_END" if count15
0x38 | F654 | 1111 0110010 10100 | goto "BR2_TEST" activate c19; counter increment

// --- BR2_END (Address 0x39) ---
0x39 | 0015 | 0000 0000000 10101 | activate c20; X/Y <- Q[15:0]
0x3A | F020 | 1111 0000001 00000 | goto "FETCH"
```

```
1111000000100000
0000000000001100
0000000000011011
0000000000001001
1111000000100000
0000000000011011
0000000000000011
0000000000011100
0000000000001110
1111000000100000
0000000000010000
0101011010100000
0110011011000000
1111011011110011
1111011010010001
1111011010010010
0111011100100000
1111011001010100
0000000000010101
1111000000100000
```

Each microinstruction is 16 bits. It contains 4-bit condition select (CS),

7-bit branch address (BA) and a 5-bit control field (CF). This results in a memory with 2^7 locations, each location being 16 bits.

128 x 2B = 256Bytes = 2048 bits = 2Kb

Memory

```
microcode_rom #(7, 16, "microcode.bin") ucode_rom (
    .addr(UPC_out),
    .data(microinstruction)
);

// Microinstruction Fields
wire [3:0]  CS = microinstruction[15:12];
wire [6:0]  BA = microinstruction[11:5];
wire [4:0]  CF = microinstruction[4:0];
```

The microcode memory is instantiated in the control unit. The address is the microprogram counter, and the output is the microinstruction.

```
assign c[0]    = dec_out[1];                          // c0
assign c[1]    = dec_out[2] | dec_out[26] | dec_out[27];// c1
assign c[4:2]  = dec_out[5:3]; // c2, c3, c4, c5
assign c[5]    = dec_out[6] | dec_out[28]; // c5
assign c[6]    = dec_out[7] | dec_out[26]; // c6
assign c[7]    = dec_out[8] | dec_out[28];     // c7
assign c[8]    = dec_out[9] | dec_out[29]; // c8
assign c[11:9] = dec_out[12:10];           // c9, c10, c11
assign c[12]   = dec_out[13] | dec_out[27];// c12
assign c[13]   = dec_out[14];
assign c[14]   = dec_out[15] | dec_out[29];
assign c[24:15] = dec_out[25:16];
```

The control field is only 5 bits and the CPU needs 25 control signals, therefore the control field is encoded vertically. The control field is decoded by a 5 to 32 decoder, however, since not all control signals are mutually exclusive, some OR gates were needed on the output of the decoder.

Signals that activate simultaneously:

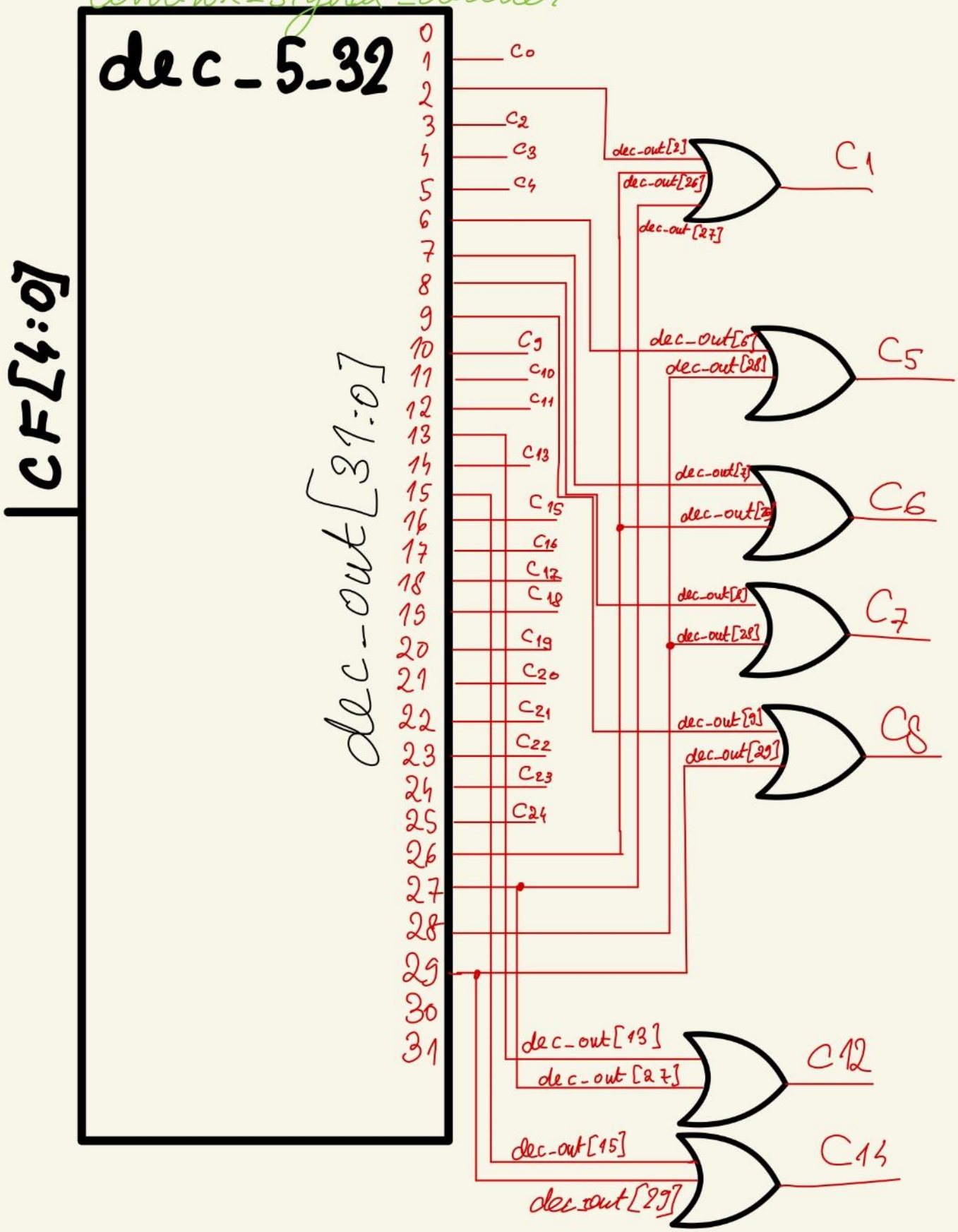
c1, c6 (c1 loads Address Register, c6 selects AR input to be DR [address]) : AR <- DR[address]

c1, c12 (c12 active means AR receives Stack Pointer as input) : AR <- StackPointer

c5, c7 (c5 - load general purpose registers X/Y, c7 select DR as input) : X/Y <- DR

c8, c14 (c8 – write to memory, c14 – select PC as input of memory) - used for jumping to a subroutine (push PC to stack)

control_signal_decoder



Instruction Decode

Instruction Decode works by setting the CS to a specific value (1000 b) , when the control unit sees this value it loads the MicroProgramCounter with the start address of the specific instructions microcode using a ROM indexed by opcode:

```
08 // 0x00 opcode 000000 LOAD
0C // 0x01 opcode 000001 STORE
05 // 0x02 opcode 000010 MOV
0F // 0x03 opcode 000011 BRA (Branch Always)
1C // 0x04 opcode 000100 JMP (call procedure)
22 // 0x05 opcode 000101 RET (Return)
28 // 0x06 opcode 000110 PUSH
2C // 0x07 opcode 000111 POP
12 // 0x08 opcode 001000
12 // 0x09 opcode 001001
12 // 0x0A opcode 001010
12 // 0x0B opcode 001011
```

Ex: 08h is the LOAD_OPERATION microcode start address. 12h is the ILLEGAL_INSTUCTION microcode address, resulting in the CPU going into an infinite loop.

// --- ILLEGAL_INSTRUCTION (Address 0x12) ---

0x12 | F259 | 1111 0010010 11001 | goto "ILLEGAL_INSTRUCTION" activate c24

```
// --- RESET (Address 0x00) ---
0x00 | 0001 | 0000 0000000 00001 | activate c0                ; PC <- 0

// --- FETCH (Address 0x01) ---
0x01 | 0002 | 0000 0000000 00010 | activate c1                ; AR <- PC
0x02 | 0003 | 0000 0000000 00011 | activate c2                ; DR <- Mem[AR]
0x03 | 0004 | 0000 0000000 00100 | activate c3                ; IR <- DR[opcode], PC <- PC+1

// --- DECODE (Address 0x04) ---
0x04 | 8000 | 1000 0000000 00000 | if DECODE                  ;Sets CS to 1000 in order to load U

// --- ALU_OPERATION (Address 0x05) ---
0x05 | 0005 | 0000 0000000 00101 | activate c4                 ; AC <- ALU_output
0x06 | 0006 | 0000 0000000 00110 | activate c5                 ; X or Y <- AC
0x07 | F020 | 1111 0000001 00000 | goto "FETCH"

// --- LOAD_OPERATION (Address 0x08) ---
0x08 | 001A | 0000 0000000 11010 | activate state26            ; AR <- DR[address]
0x09 | 0003 | 0000 0000000 00011 | activate c2                 ; DR <- Mem[AR]
0x0A | 001C | 0000 0000000 11100 | activate state28             ; X or Y <- DR
0x0B | F020 | 1111 0000001 00000 | goto "FETCH"
```

Instruction Set

Hex Opcode	Binary Opcode	Mnemonic	Category	Description
0x00	0	LOAD	Data Transfer	Load value from memory to register
0x01	1	STORE	Data Transfer	Store value from register to memory
0x02	10	MOV	Data Transfer	Move data between registers
0x03	11	BRA	Control Flow	Branch Always (Unconditional)
0x04	100	JMP	Control Flow	Jump to procedure (Call)
0x05	101	RET	Control Flow	Return from procedure
0x06	110	PUSH	Stack	Push value onto the stack
0x07	111	POP	Stack	Pop value from the stack
0x10	10000	BRZ	Branch (Cond)	Branch if Zero
0x11	10001	BRN	Branch (Cond)	Branch if Negative
0x12	10010	BRC	Branch (Cond)	Branch if Carry
0x13	10011	BRO	Branch (Cond)	Branch if Overflow
0x20	100000	ADD	Arithmetic	Addition
0x21	100001	SUB	Arithmetic	Subtraction
0x22	100010	MUL	Arithmetic	Multiplication
0x23	100011	DIV	Arithmetic	Division
0x24	101100	MOD	Arithmetic	Modulo (Remainder)
0x25	100101	INC	Arithmetic	Increment value by 1
0x26	100110	DEC	Arithmetic	Decrement value by 1
0x27	100111	CMP	Logic/Comparisc	Compare two values (sets flags)
0x30	110000	AND	Logical	Bitwise AND
0x31	110001	OR	Logical	Bitwise OR
0x32	110010	XOR	Logical	Bitwise XOR
0x33	110011	NOT	Logical	Bitwise NOT (Invert)
0x34	110100	TST	Logic/Comparisc	Test bits (Logical AND, sets flags)
0x38	111000	LSL	Shift/Rotate	Logical Shift Left
0x39	111001	LSR	Shift/Rotate	Logical Shift Right
0x3A	111010	RSL	Shift/Rotate	Rotate Shift Left
0x3B	111011	RSR	Shift/Rotate	Rotate Shift Right

FPGA Demo – Memory Mapped I/O

- 7-segment display mapped to the memory location 0x1FE

- 6 leds are mapped to the memory location 0x1FF

Fibonacci Numbers – on 7 seg display

```
MOV X, 0
    STORE X, 0x100
    MOV X, 1
    STORE X, 0x101

main_loop:
    LOAD X, 0x101
    STORE X, 0x1FE

    LOAD X, 0x100
    LOAD Y, 0x101

add_loop:
    TST Y, 511
    BRZ end_add
    ADD X, 1
    SUB Y, 1
    BRA add_loop

end_add:
    LOAD Y, 0x101
    STORE Y, 0x100
    STORE X, 0x101
    BRA main_loop
```

The processor does not have ADD X, Y instruction; it can only add an immediate value to a general-purpose register. To calculate the sum of 2 registers, you must repeatedly add the value 1 to X, Y times. This is what

the code above does => each time a new Fibonacci number is calculated it takes longer.

The code is inefficient O(2^n) time complexity

Test stack and multiplication

```
# No. | Hex Value | Decimal Value
# -------------------------------
# 1   | 0x4ce3    | 19683
# 2   | 0x19a1    | 6561
# 3   | 0x088b    | 2187
# 4   | 0x02d9    | 729
# 5   | 0x00f3    | 243
# 6   | 0x0051    | 81
# 7   | 0x001b    | 27
# 8   | 0x0009    | 9
# 9   | 0x0003    | 3
# 10   | 0x0001    | 1
```

```
    MOV Y, 10
    MOV X, 1

PUSH_LOOP:
    PUSH X
    MUL X, 3
    DEC Y, 0
    BRZ POP_START
    BRA PUSH_LOOP

POP_START:
    MOV Y, 10
POP_LOOP:
    POP X
    STORE X, 0x1FE
    DEC Y, 0
    BRZ EXIT
    BRA POP_LOOP

EXIT:
    BRA EXIT
```